Arturo Ordaz-Gutierrez

CSC 369: Introduction to Distributed Computing

California Polytechnic State University, San Luis Obispo

June 2026

Spam Email Classification Using Bag of Words, TF-IDF, and KNN w/ Scala & Spark

Introduction

This project implements a spam email classifier using Spark and Scala. The project uses three techniques implemented from scratch using only RDDs. Bag of Words (BoW) for raw word count features, Term Frequency Inverse Document Frequency (TF-IDF) for weighted features, and K-Nearest Neighbors (KNN) for classification using cosine similarity which is better for text. The implementations themselves were written from scratch using Spark RDD operations, with no machine learning libraries used.

I am trying to compare two feature representations, Bag of Words and TF-IDF, by running both through the same KNN classifier and outputting which produces better accuracy, precision, recall, and F1 scores when predicting whether an email is Spam or Ham (Not).

Dataset

The dataset is a collection of 193,852 emails from Kaggle containing two columns: a label (Spam or Ham) and the email text. It contains roughly 91,000 spam and 102,000 ham emails. The raw CSV spans 889,000 lines because email bodies contain newlines and commas within fields.

Preprocessing

Each email is scanned through using a parser that handles quoted fields, then cleaned by converting to lowercase and removing special characters. A small stopword list filters out HTML

artifacts commonly found in email bodies (img, href, nbsp, div, span, etc.) since these carry no meaning. Common English words are kept in the vocabulary, since their presence or absence emails can still contribute some information especially when comparing BoW and TF-IDF. The dataset is shuffled and split 80/20 into training and test RDDs, both in memory.

Bag of Words

Bag of Words represents each email as a map of word. The code follows the MapReduce word count pattern from class. flatMap returns ((docId, word), 1.0) pairs for every word in each email, reduceByKey sums identical keys across the cluster, and groupByKey reassembles per-document word count maps. The vectors are joined with the email RDD to attach spam/ham labels.

TF-IDF

TF-IDF improves on BoW by weighting words according to how uncommon they are. Words that are frequent in one email but rare across multiple receive a higher "importance" score. It is computed in three steps:

Term Frequency: word counts are computed using the same flatMap & reduceByKey pattern as BoW, then normalized by dividing each count by the maximum count in that document.

Document Frequency: flatMap emits (word, 1.0) for each unique word per document, reduceByKey sums across all documents, and collectAsMap brings the result to the driver.

TF-IDF: IDF is computed as

$$\log\left(\frac{N}{DF}\right)$$

for each word. The IDF map is distributed to all nodes using a broadcast variable so each node

can look up IDF values locally rather than shipping the map with every task. Each word's TF is multiplied by its IDF to produce the final vector.

KNN with Cosine Similarity

KNN classifies each test email by finding the k most similar training emails and taking a majority vote on their labels. Similarity is measured using cosine similarity: the dot product of two vectors divided by the product of their magnitudes. Only words present in both vectors contribute, making it efficient for mostly empty text vectors.

$$\cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|}$$

The training vectors are collected and distributed using a broadcast variable. The test RDD is processed using mapPartitions, which loads the broadcast array once per partition rather than once per test email, reducing the setup cost. For each test email, cosine similarity is computed against every training email, the top k are selected, and majority vote determines the prediction.

Results

The classifier was run on 8,000 emails (3,779 spam, 4,221 ham) with k=5 and a vocabulary of 76,443 unique words.

Classification Results (k = 5, 8,000 emails)

Method	Accuracy	Precision	Recall	F1
Bag of Words	0.7073	0.6609	0.7318	0.6945
TF-IDF	0.9074	0.9026	0.8927	0.8976

Confusion Matrices

Method	TP	FP	TN	FN
Bag of Words	532	273	599	195
TF-IDF	649	70	802	78

TF-IDF outperforms Bag of Words across every metric, achieving 90.74% accuracy versus 70.73% for BoW. The biggest difference is in false positives: TF-IDF produces only 70 compared to 273 for BoW. This is because TF-IDF adds less importance to common words that appear in both spam and ham, and adds more importance to unique words, giving KNN better signal to classify on.

What I Learned

The MapReduce word count pattern from lecture (flatMap to emit pairs, reduceByKey to sum) is the exact foundation for both BoW and TF-IDF. Broadcast variables were essential for distributing the IDF lookup table and training vectors efficiently. Using mapPartitions in KNN reduced overhead by loading data once per partition instead of per element. The comparison showed that feature representation matters as much as the classification algorithm, TF-IDF's 17-point accuracy improvement over BoW comes entirely from weighting words by importance.

Major Obstacles

Parsing the CSV was difficult because email bodies contain newlines, commas, and quotes that break standard parsers. My parser had to track quoted fields across multiple lines. Memory management was a constant challenge. KNN's pairwise similarity computation scales quadratically with dataset size. Running the full 193,852 email dataset caused the program to timeout, so the final results use an 8,000-email sample that still demonstrates the distributed pipeline and produces meaningful accuracy comparisons.

Conclusion

This project built a spam classifier from scratch using only Spark RDDs, comparing Bag of Words and TF-IDF features through a KNN classifier with cosine similarity. TF-IDF achieved 90.74% accuracy versus 70.73% for BoW on 8,000 emails. The project demonstrates that distributed computing techniques from CSC 369, flatMap, reduceByKey, mapPartitions, and join, apply directly to real machine learning problems.

References

Alexander, A. "An Apache Spark word count example (Scala)." *Scala Cookbook.* https://alvinalexander.com/source-code/scala-cookbook-apache-spark-word-count-example/

Kanaparthy, V. (2015). "Simple document classification using cosine similarity on Spark." https://venukanaparthy.wordpress.com/2015/08/16/simple-document-classification-using-cosine-similarity-on-spark/

Kaggle. "190K+ Spam | Ham Email Dataset for Classification" https://www.kaggle.com/datasets/meruvulikith/190k-spam-ham-email-dataset-for-classification